

Busca em Árvores Binárias

Árvore Binária de Busca

□ Definição

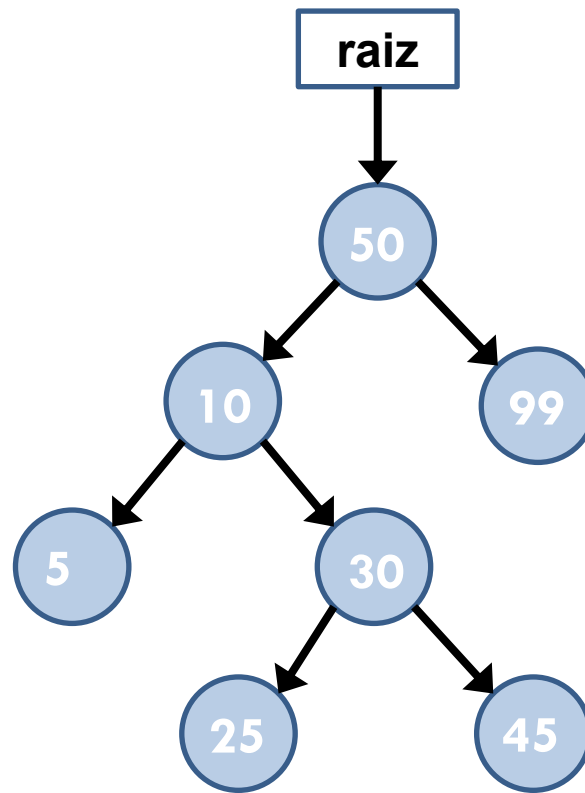
- ▣ É uma árvore binária
 - Cada nó pode ter 0, 1 ou 2 filhos
- ▣ Cada nó possui da árvore possui um valor (chave) associado a ele
 - Não existem valores repetidos
- ▣ Esse valor determina a posição do nó na árvore

Árvore Binária de Busca

- Regra para posicionamento dos valores na árvore
 - ▣ Para cada nó pai
 - todos os valores da sub-árvore **esquerda são menores** do que o nó pai
 - todos os valores da sub-árvore **direita são maiores** do que o nó pai;
 - ▣ Inserção e remoção devem ser realizadas respeitando essa regra de posicionamento dos nós.

Árvore Binária de Busca

□ Exemplo



Árvore Binária de Busca

- Ótima alternativa para operações de busca binária
 - ▣ Possui a vantagem de ser uma estrutura dinâmica em comparação ao array
 - ▣ É mais fácil inserir valores na árvore do que em um array ordenado
 - Array: envolve deslocamento de elementos

Árvore Binária de Busca

- Custo para as principais operações em uma **árvore binária de busca** contendo **N** nós.
 - ▣ O pior caso ocorre quando a árvore não está balanceada

	Melhor Caso	Pior Caso
Inserção	$O(\log N)$	$O(N)$
Remoção	$O(\log N)$	$O(N)$
Busca	$O(\log N)$	$O(N)$

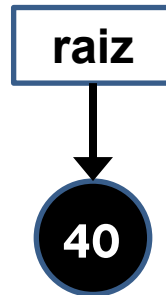
Árvore Binária de Busca - Inserção

- Para inserir um valor **V** na árvore
 - ▣ Se a raiz é igual a **NULL**, insira o nó
 - ▣ Se **V** é menor do que a raiz: vá para a **sub-árvore esquerda**
 - ▣ Se **V** é maior do que a raiz: vá para a **sub-árvore direita**
 - ▣ Aplique o método **recursivamente**
 - pode ser feito sem recursão
- Dessa forma, percorremos um conjunto de nós da árvore até chegar ao nó folha que irá se tornar o pai do novo nó

Árvore Binária de Busca: Inserção

- Devemos também considerar a inserção em uma árvore que está vazia

Inserir o valor '40'
em uma árvore
vazia



```
*raiz = novo;
```


TAD Árvore Binária

□ Inserção em uma Árvore Binária de Busca

```
int insere_ArvBin(ArvBin* raiz, int valor){  
    if(raiz == NULL)  
        return 0;  
    struct NO* novo;  
    novo = (struct NO*) malloc(sizeof(struct NO));  
    if(novo == NULL)  
        return 0;  
    novo->infor = valor;  
    novo->dir = NULL;  
    novo->esq = NULL;  
    //procurar onde inserir!
```

TAD Árvore Binária

□ Inserção em uma Árvore Binária de Busca

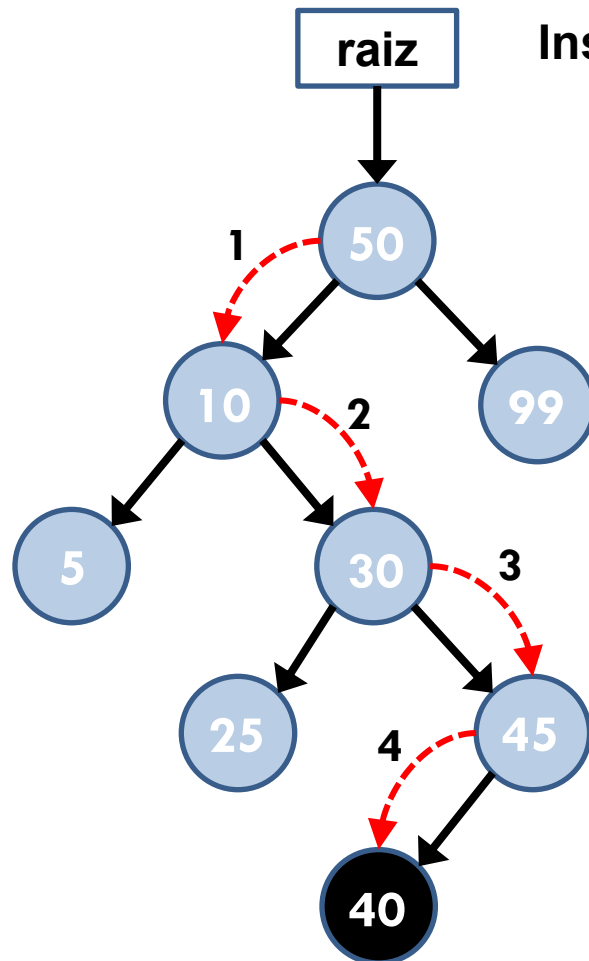
Navega nos nós
da árvore até
chegar em um nó
folha

Insere como filho
desse nó folha

```
if(*raiz == NULL)
    *raiz = novo;
else{
    struct NO* atual = *raiz;
    struct NO* ant = NULL;
    while(atual != NULL){
        ant = atual;
        if(valor == atual->info){
            free(novo);
            return 0; //elemento já existe
        }
        if(valor > atual->info)
            atual = atual->dir;
        else
            atual = atual->esq;
    }
    if(valor > ant->info)
        ant->dir = novo;
    else
        ant->esq = novo;
}
return 1;
```

Árvore Binária de Busca: Inserção

Exemplo



Inserir o valor '40' como um nó folha

1	valor é menor do que 50: visita filho da esquerda
2	valor é maior do que 10: visita filho da direita
3	valor é maior do que 30: visita filho da direita
4	valor é menor do que 45: visita filho da esquerda
Fim	Não existe filho da esquerda. Valor passa a ser o filho da esquerda de 45

Árvore Binária de Busca: Busca

- Consultar se um determinado nó **V** existe em uma árvore é similar a operação de inserção
 - ▣ primeiro compare o valor buscado com a **raiz**;
 - ▣ se **V** é menor do que a raiz: vá para a **sub-árvore da esquerda**;
 - ▣ se **V** é maior do que a raiz: vá para a **sub-árvore da direita**;
 - ▣ aplique o método recursivamente até que a raiz seja igual ao valor buscado
 - pode ser feito sem recursão

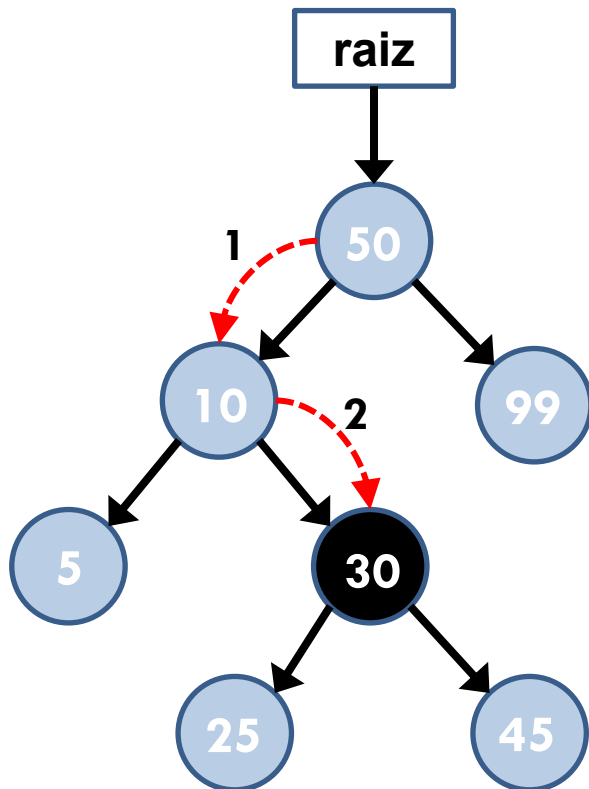
TAD Árvore Binária

□ Busca em uma Árvore Binária de Busca

```
6  int consulta_ArvBin(ArvBin *raiz, int valor){
7      if(raiz == NULL)
8          return 0;
9      struct NO* atual = *raiz;
10     while(atual != NULL){
11         if(valor == atual->info){
12             return 1;
13         }
14         if(valor > atual->info)
15             atual = atual->dir;
16         else
17             atual = atual->esq;
18     }
19     return 0;
20 }
```

Árvore Binária de Busca: Busca

Exemplo

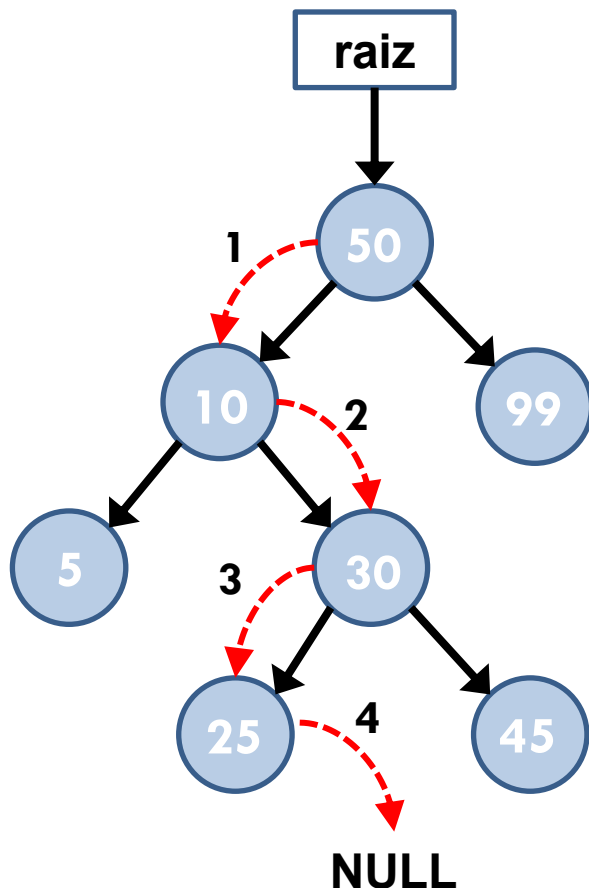


Valor procurado: 30

1	valor procurado é menor do que 50: visita filho da esquerda
2	valor procurado é maior do que 10: visita filho da direita
Fim	valor procurado é igual ao do nó: retornar dados do nó

Árvore Binária de Busca: Busca

□ Exemplo: valor buscado não existe!



Valor procurado: 28

1	valor procurado é menor do que 50: visita filho da esquerda
2	valor procurado é maior do que 10: visita filho da direita
3	valor procurado é menor do que 30: visita filho da esquerda
4	valor procurado é maior do que 25: visita filho da direita
Fim	Filho da direita de 25 não existe: a busca falhou

Árvore Binária de Busca: Remoção

- Remover um nó de uma árvore binária de busca não é uma tarefa tão simples quanto a inserção.
 - ▣ Isso ocorre porque precisamos procurar o nó a ser removido da árvore o qual pode ser um
 - nó folha
 - nó interno (que pode ser a raiz), com um ou dois filhos.
 - ▣ Se for um nó interno
 - Reorganizar a árvore para que ela continue sendo uma árvore binária de busca

TAD Árvore Binária

- Remoção em uma Árvore Binária de Busca
 - ▣ Trabalha com 2 funções
 - Busca pelo nó
 - Tratar os 3 tipos de remoção: com 0, 1 ou 2 filhos

```
8  int remove_ArvBin(ArvBin *raiz, int valor){
9      /*
10     FUNÇÃO RESPONSÁVEL PELA BUSCA
11     DO NÓ A SER REMOVIDO
12     */
13 }
14 struct NO* remove_atual(struct NO* atual){
15     /*
16     FUNÇÃO RESPONSÁVEL POR TRATAR OS 3
17     TIPOS DE REMOÇÃO
18     */
19 }
```

TAD Árvore Binária

□ Remoção em uma Árvore Binária de Busca

Achou o nó a ser removido. Tratar o tipo de remoção

Continua andando na árvore a procura do nó a ser removido

```
int remove_ArvBin(ArvBin *raiz, int valor){
    if(raiz == NULL) return 0;
    struct NO* ant = NULL;
    struct NO* atual = *raiz;
    while(atual != NULL){
        if(valor == atual->info){
            if(atual == *raiz)
                *raiz = remove_atual(atual);
            else{
                if(ant->dir == atual)
                    ant->dir = remove_atual(atual);
                else
                    ant->esq = remove_atual(atual);
            }
            return 1;
        }
        ant = atual;
        if(valor > atual->info)
            atual = atual->dir;
        else
            atual = atual->esq;
    }
    return 0;
}
```

TAD Árvore Binária

Remoção em uma Árvore Binária de Busca

```
1 struct NO* remove_atual(struct NO* atual) {
2     struct NO *no1, *no2;
3     if(atual->esq == NULL) {
4         no2 = atual->dir;
5         free(atual);
6         return no2;
7     }
8     no1 = atual;
9     no2 = atual->esq;
10    while(no2->dir != NULL) {
11        no1 = no2;
12        no2 = no2->dir;
13    }
14
15    if(no1 != atual) {
16        no1->dir = no2->esq;
17        no2->esq = atual->esq;
18    }
19    no2->dir = atual->dir;
20    free(atual);
21    return no2;
22 }
```

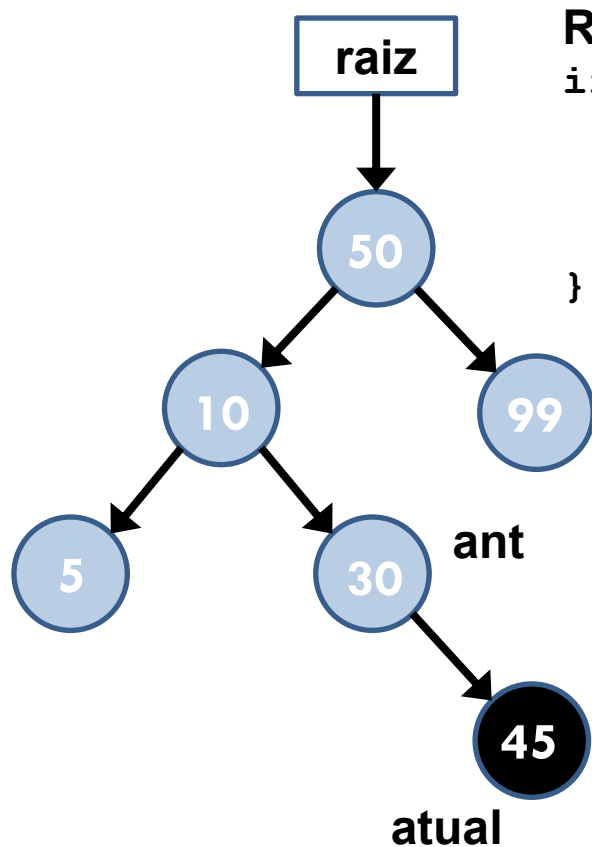
Sem filho da esquerda.
Apontar para o filho da direita (trata nó folha e nó com 1 filho)

Procura filho mais a direita na sub-árvore da esquerda.

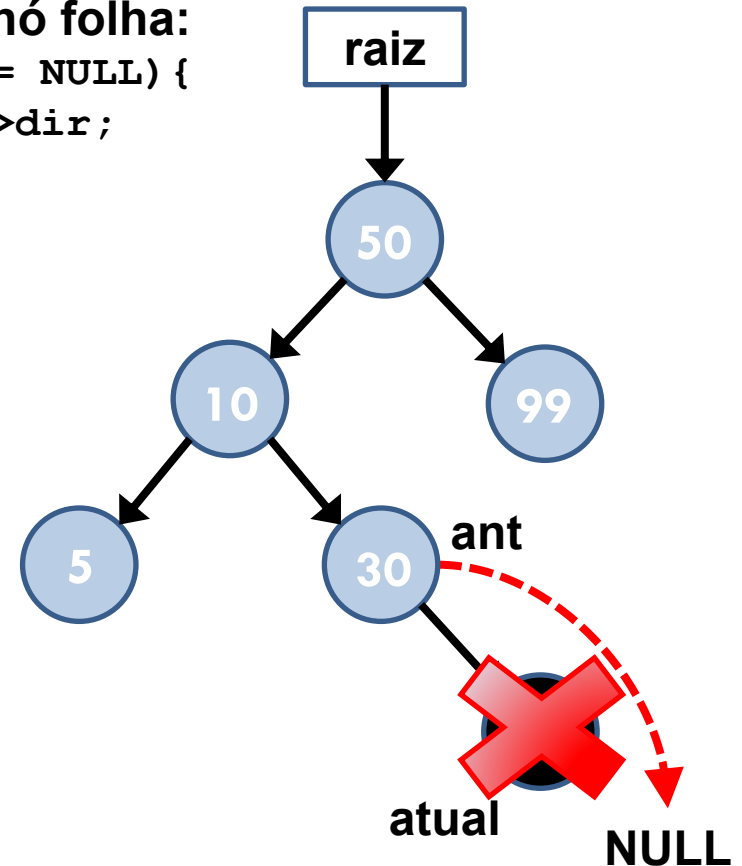
Copia o filho mais a direita na sub-árvore da esquerda para o lugar do nó removido.

Árvore Binária de Busca: Remoção

□ Exemplo: remoção de um nó folha

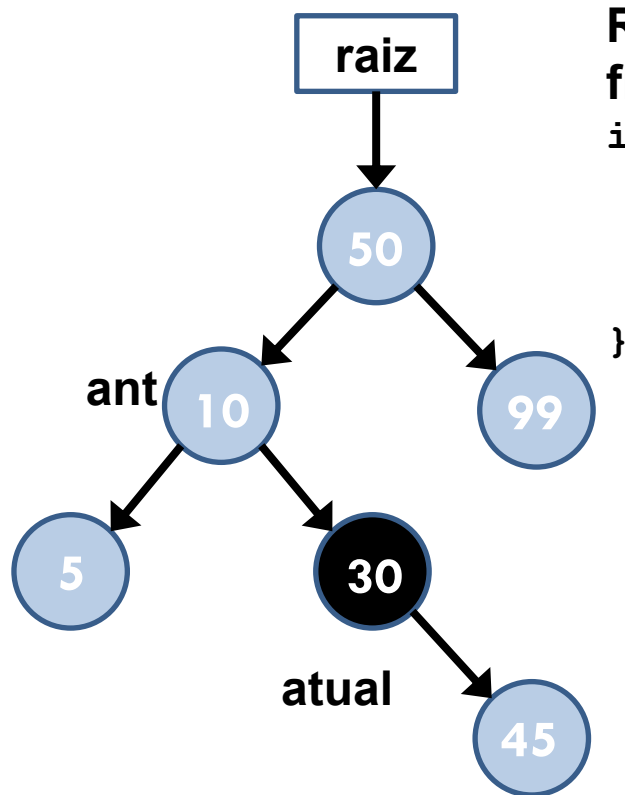


Remoção de um nó folha:
`if (atual->esq == NULL) {
 no2 = atual->dir;
 free(atual);
 return no2;
}`



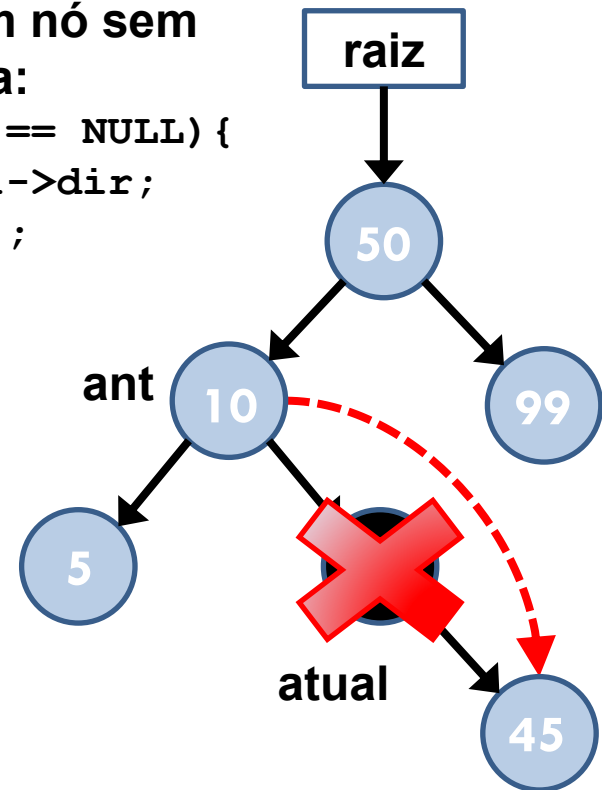
Árvore Binária de Busca: Remoção

- Exemplo: remoção de um nó com 1 filho



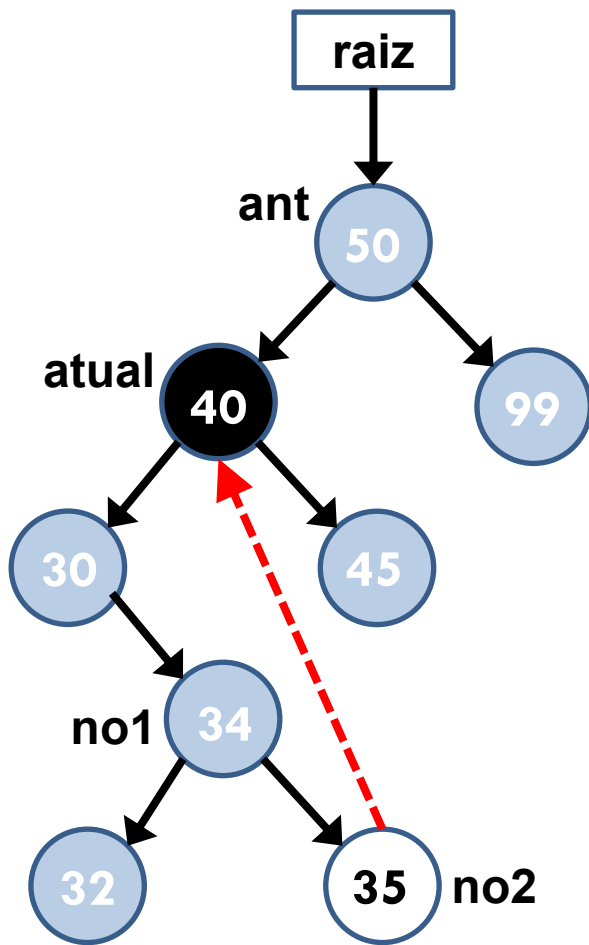
Remoção de um nó sem
filho a esquerda:

```
if(atual->esq == NULL){  
    no2 = atual->dir;  
    free(atual);  
    return no2;  
}
```



Árvore Binária de Busca: Remoção

- Exemplo: remoção de um nó com 2 filhos



Remoção de um nó
com ambos os
filhos:

Remover o nó
“atual” e substituí-lo
pelo nó “mais a
direita” da sub-
árvore da esquerda
do nó “atual”

